CourtDecisionBaseline: описательный legacy-гайд структуры MVP-моделей судебного решения

Содержание

1 Назначение **2**

2 Основные файлы **2**
2.1 Общая схема обработки данных . . . 3
2.2 Фильтрация и извлечение текстовых признаков . . . 4

3 Какие данные подаются на вход **4**

4 Признаки обстоятельств **4**

5 Регуляторные признаки **5**
5.1 Выбор профиля и формирование признаков . . . 5
5.2 Точки подключения к модели . . . 6
5.3 Смысл и границы постобработки . . . 6

6 Таргеты судебного решения **7**

7 Разбиение на train, val и test **7**

8 Методы моделей **7**
8.1 Independent . . . 8
8.2 Chain . . . 8
8.3 Graph . . . 9
8.4 NLP baseline . . . 10
8.5 MLP . . . 11

9 Балансировка классов и статей **12**

10 Метрики качества **12**
10.1 Target score . . . 12
10.2 `test_decision_score` . . . 13
10.3 `test_balanced_article_score` . . . 13
10.4 `test_substantive_score` . . . 13
10.5 `test_article_score_gap` . . . 13
10.6 `legally_constrained_decision_score` . . . 13

11 Текущие результаты **14**
11.1 Полный сохранённый baseline результаты для independent . . . 14

12 Основные проблематики **14**
12.1 Разная природа статей . . . 14
12.2 Регуляторная согласованность . . . 14
12.3 Работа с реальным текстом . . . 15

13 Экспериментальное тестирование методов и способов балансировки **15**
13.1 Конфигурация эксперимента . . . 15
13.2 Состав и дисбаланс данных . . . 15
13.3 Сравнение методов без балансировки . . . 16
13.4 Полная матрица балансировки . . . 16
13.5 Результаты по статьям . . . 17
13.6 Результаты способов балансировки . . . 17
13.7 Итоговый вывод . . . 18

1 Назначение

`court_decision_baseline.py` - основной MVP-код для обучения моделей судебного решения по уже собранным числовым векторам дел.

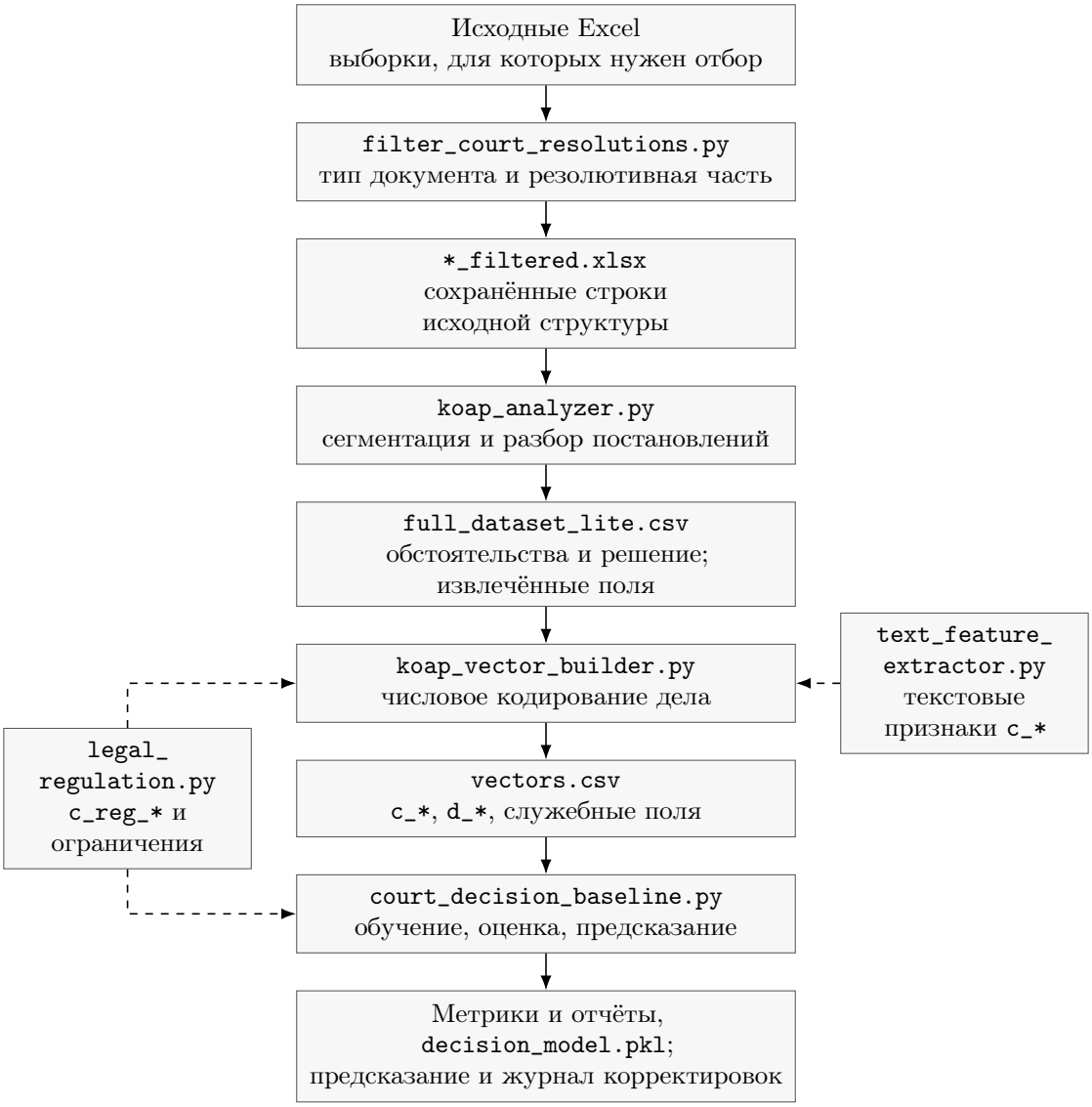
Задача модели: получить на вход вектор обстоятельств дела и на выходе выдать вектор судебного решения. Обработка включает подготовку исходных документов, выделение обстоятельств и решения, построение числовых векторов, обучение и оценку моделей.

2 Основные файлы

Файл	**Роль**
`filter_court_resolutions.py`	Предварительный отбор строк Excel по типу документа и признакам исхода по целевым статьям в резолютивной части. Самостоятельный этап подготовки данных.
`koap_analyzer.py`	Разбор текстов, выделение обстоятельств и решения, экспорт облегчённой таблицы и таблицы с полным текстом.
`text_feature_extractor.py`	Библиотека правил извлечения обстоятельств из текста: освидетельствование, документы, признание вины, история наказаний и другие признаки. Вызывается векторизатором.
`koap_vector_builder.py`	Преобразование результатов разбора в `c_*`, `d_*` и служебные поля; добавление параметров нормы; сохранение `vectors.csv`.
`legal_regulation.py`	Встроенный реестр правовых профилей, признаки `c_reg_*`, ограничение предсказаний диапазонами профиля и журнал корректировок.
`court_decision_baseline.py`	Обучение, сравнение, диагностика и предсказание судебных решений; оценка исходного и скорректированного результата.

2.1 Общая схема обработки данных

Сплошные стрелки показывают передачу данных, пунктирные - использование вспомогательного модуля. Каждый этап подготовки запускается отдельно.



Анализатор также сохраняет `full_dataset_heavy.csv` с полным текстом. На схеме показан основной путь через lite-таблицу. Готовый `vectors.csv` можно сразу передать модели.

2.2 Фильтрация и извлечение текстовых признаков

`filter_court_resolutions.py` нормализует пробелы, регистр и HTML-разметку; при включённой проверке типа документа оставляет постановления. Затем `extract_resolution` выделяет текст от первого маркера резолюции, а `find_article_outcomes` ищет упоминания 12.8 или 264.1 рядом с индикаторами виновности, оправдания или прекращения за отсутствием состава либо события.

Векторизатор создаёт `TextFeatureExtractor` и вызывает его из `VectorBuilder._extract_text_features`. В текущем коде для этого объединяются `circumstances` и `court_decision`. Регулярные выражения и правила возвращают числовые либо логические значения. Эти значения входят в `c_*`; целевые `d_*` векторизатор собирает отдельно из наказания и текста решения.

Извлекатель является библиотекой правил, а TF-IDF в режиме `nlp` представлением уже собранных признаков. Они выполняют разные этапы обработки.

3 Какие данные подаются на вход

На вход обучения подаются один или несколько `vectors.csv`. Пример:

```
python court_decision_baseline.py train \
  --data "путь до 12.8" "путь до 264.1" \
  --method independent \
  --balance none \
  --out court_decision_independent_none
```

Текущий основной набор для baseline:

Источник	**Смысл**
`koap_vectors/12.8 Сургут/overall/vectors.csv`	дела по статье 12.8, Сургут
`koap_vectors/264.1 суды общей инстанции/vectors.csv`	дела по статье 264.1

В сохранённом полном запуске использовалось 90+ тысяч строк:

Статья	**Количество строк**
12.8	70 000+
264.1	10 000+

4 Признаки обстоятельств

Модель автоматически берёт как входные признаки все колонки, начинающиеся с `c_`.

Группа	Примеры	Смысл
Норма и суд	`c_article_code`, `c_article_part`, `c_court_type`	статья, часть статьи, тип суда
Время	`c_year`, `c_month`, `c_day_of_week`, `c_hour`, `c_is_night`	временной контекст события
География	`c_region_code`, `c_city_code`	регион и город
История лица	`c_recidivism_count`, `c_fine_history`, `c_license_history`	повторность, история штрафов и лишения прав
Процесс	`c_has_lawyer`, `c_is_appeal`, `c_guilt_admitted`, `c_sobriety_test`	адвокат, апелляция, признание вины, освидетельствование
Доказательства	`c_has_protocol`, `c_has_witness_testimony`, `c_has_officer_report`, `c_has_device_cert`	процессуальные документы и доказательства
Условия дела	`c_vehicle_ok`, `c_is_work_hours`, `c_special_procedure`	исправность автомобиля, рабочее время, особый порядок
Участники	`c_driver_injured`, `c_driver_health`, `c_victim_injured`, `c_victim_health`	статус здоровья водителя и потерпевшего
Регуляторные	`c_reg_*`	числовое описание действующего правового режима

При этом `source_case_id` не используется как признак модели. Это снижает риск переобучения на идентификаторы дел.

5 Регуляторные признаки

Регуляторный блок нужен, чтобы модель учитывала, какие санкции были возможны в конкретный год и по конкретной статье.

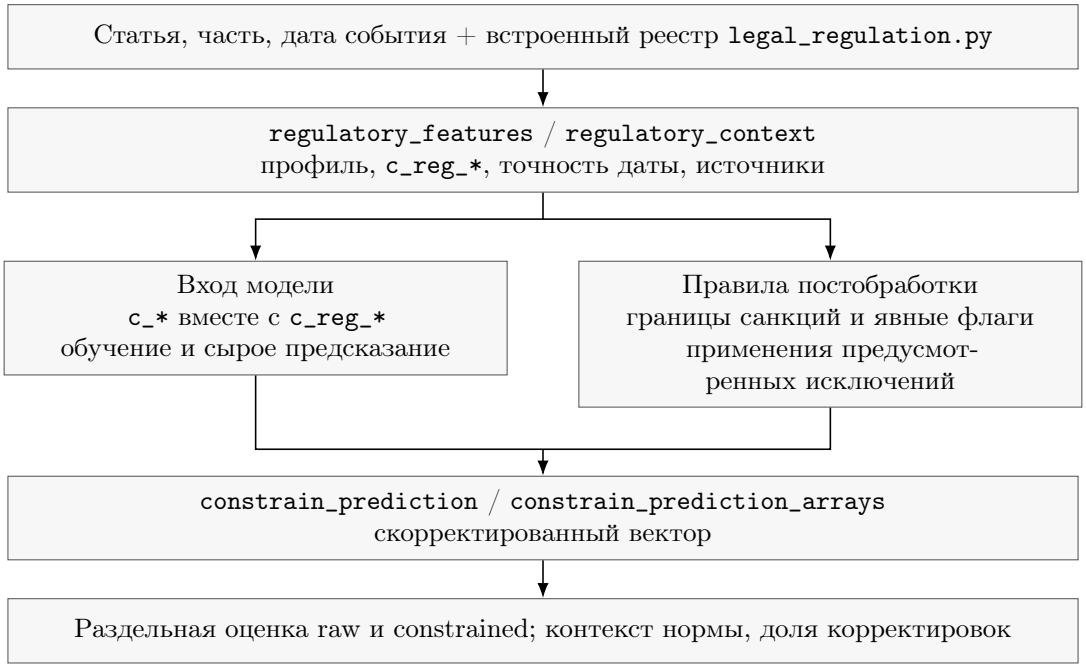
5.1 Выбор профиля и формирование признаков

Функция `regulatory_features(article_code, year, part, month, day)` получает код статьи, часть и дату события. Коды 128 и 2641 соответствуют двум поддерживаемым статьям. Год, месяц и день берутся из `c_year`, `c_month`, `c_day`; при неполной дате может использоваться `crime_datetime_iso`. По этим значениям выбираются временной профиль и набор числовых параметров.

Признаки включают код и дату начала профиля, точность даты, тип ответственности, границы штрафа и сроков наказаний. Например, `c_reg_fine_min_rub` и `c_reg_fine_max_rub` задают денежный диапазон, а `c_reg_license_revocation_min_months` и `c_reg_license_revocation_max_months` - диапазон срока лишения права в месяцах. Это параметры встроенного профиля, а не статистики наказаний из обучающей выборки.

Если известен только год или месяц, код выбирает профиль на начало известного интервала. При наличии изменения нормы внутри этого интервала выставляется `c_reg_date_ambiguous = 1`. Сам флаг не блокирует последующую корректировку; поэтому неоднозначные даты нужно отдельно анализировать. Дата вынесения реше-

ния не является предсказываемым таргетом текущей модели. При этом векторизатор может использовать исходный `year`, если дата события не извлечена: семантику этого года необходимо проверять в источнике.



5.2 Точки подключения к модели

1. `CourtVector.to_flat_dict` в векторизаторе вызывает `regulatory_features` и записывает `c_reg_*` в `vectors.csv`. Нормативный год `c_article_year_norm` уточняется по выбранному профилю.
2. В модели `load_training_data` добавляет параметры через `enrich_regulatory_features`. Метод `DecisionModelSystem.fit` проверяет наличие колонок и совпадение версии. При необходимости признаки пересчитываются. Они входят в общий вход `c_*` для всех режимов моделей.
3. `DecisionModelSystem.evaluate` получает сырые предсказания и вызывает `constrain_prediction_arrays`. Основные метрики считаются по сырым значениям, а `legally_constrained_decision_score` - по скорректированным.
4. `predict_row_details` возвращает `raw_prediction`, `constrained_prediction`, `legal_context`, `constraint_audit` и `input_features`. Более короткий метод `predict_row` возвращает только скорректированный вектор.

5.3 Смысл и границы постобработки

Сроки `d_license_revocation_epochs` и `d_imprisonment_epochs` перед проверкой переводятся в единицы нормы: в коде одна эпоха соответствует шести месяцам, а для административного ареста используется пересчёт через 180 суток. Обратно в предсказание записываются эпохи. Денежные значения проверяются в рублях, дополнительные виды работ - в часах или месяцах соответствующего таргета.

При неподдерживаемой статье или отсутствующем годе `constrain_prediction` возвращает исходное предсказание без корректировок. Отсутствие изменений поэтому не доказывает правовую корректность результата.

6 Таргеты судебного решения

Модель предсказывает только те `d_*`-колонки, которые есть в конкретном `vectors.csv` и входят в список поддерживаемых таргетов. В текущем полном запуске 9 таргетов:

- `d_imprisonment_epochs`
- `d_license_revocation_epochs`
- `d_conditional`
- `d_fine_rub`
- `d_travel_ban`
- `d_reconciliation`
- `d_costs_on_state`
- `d_costs_amount_rub`
- `d_self_recusal`

7 Разбиение на train, val и test

Разбиение строится стратифицированно по ключу:

`c_article_code : c_region_code : c_year`

По умолчанию:

train	=	70%
val	=	15%
test	=	15%

Индексы train, val и test не пересекаются.

8 Методы моделей

Все методы реализованы в файле `court_decision_baseline.py`. Класс `DecisionModelSystem` управляет выбором метода, обучением и оценкой. Его метод `_make_bundle` создаёт `GraphDecisionModel` для режимов `independent`, `chain` и `graph`, `NlpDecisionModel` для `nlp` и `MlpDecisionModel` для `mlp`. Название класса `GraphDecisionModel` не означает, что любой использующий его режим учитывает связи между таргетами.

Обозначим через x_i вектор обстоятельств дела i, через y_{ij} - значение таргета j, через m - количество предсказываемых таргетов. Вход формируется из колонок `c_*`, включая регуляторные `c_reg_*`. Цели выбираются из поддерживаемых колонок `d_*`. Тип задачи определяется типом таргета: например, размер штрафа требует регрессии, а признак условного наказания - бинарной классификации.

Для табличных методов функция `numeric_matrix` преобразует признаки в числовую матрицу, заменяя пропуски и нечисловые значения нулями. Пропущенные таргеты также преобразуются в ноль. Поэтому нулевое значение может объединять разные ситуации: фактическое отсутствие признака и отсутствие извлечённой информации. Это ограничение подготовки данных, которое необходимо учитывать при интерпретации качества.

Для денежных сумм и сроковиспользуется преобразование:

$$u_{ij} = \log(1 + \max(y_{ij}, 0)), \qquad \widehat{y}_{ij} = \max(0, \exp(\widehat{u}_{ij}) - 1).$$

Оно уменьшает различие масштабов между небольшими и очень большими значениями. Например, крупный штраф слабее доминирует в квадратичной ошибке после логарифмирования. При этом оптимизация ошибки логарифмов не равнозначна оптимизации MAE в рублях или месяцах. Итоговые метрики рассчитываются после обратного преобразования.

В режимах на основе `TargetEstimator` постоянный обучающий таргет обрабатывается отдельно: если все его значения одинаковы, сохраняется константа вместо обучения классификатора или регрессора. После предсказания каждого семейства регуляторный модуль отдельно формирует скорректированный результат.

8.1 Independent

Для каждого таргета обучается отдельная модель, использующая один и тот же вектор обстоятельств:

$$\widehat{y}_1 = f_1(x), \quad \widehat{y}_2 = f_2(x), \quad \ldots, \quad \widehat{y}_m = f_m(x).$$

Например, одна модель предсказывает размер штрафа, другая - срок лишения права управления, третья - условность наказания. Значения остальных таргетов не добавляются во вход. В коде это реализовано через одну `ChainModel` с параметром `use_context=False`; отдельные оцениватели хранятся в `TargetEstimator`.

Для числового таргета используется `HistGradientBoostingRegressor`, для бинарного или категориального - `HistGradientBoostingClassifier`. Градиентный бустинг последовательно строит деревья, каждое из которых корректирует результат предыдущих:

$$F_t(x) = F_{t-1}(x) + \eta h_t(x).$$

Здесь F_{t-1} - уже построенная модель, h_t - очередное дерево, η - скорость обучения. Направление корректировки определяется производными функции потерь. Для регрессии используется квадратичная ошибка, для классификации - логарифмическая потеря. В классификации сумма вкладов деревьев преобразуется в вероятности классов.

Дерево разделяет объекты условиями вида «значение признака меньше порога». Последовательность условий позволяет учитывать взаимодействия: например, различать сочетания статьи, правового периода и обстоятельств. Гистограммная реализация предварительно распределяет числовые значения по интервалам и ищет разделения между ними, сокращая количество проверяемых порогов.

Максимальная глубина дерева равна 4. Для классификации заданы 220 итераций и скорость 0.08, для регрессии - 260 и 0.06. Ранняя остановка отключена. Метод удобен как табличный baseline: он моделирует нелинейные зависимости и позволяет анализировать каждый таргет отдельно. Однако модели не согласуют свои выходы между собой. Коды категорий передаются как числа, без явного указания категориальных признаков. Слово independent относится к раздельному обучению таргетов; метод не является наивным байесовским классификатором.

8.2 Chain

Режим использует `ChainModel` с `use_context=True`. Таргеты предсказываются последовательно в порядке `DEFAULT_ORDER`; отсутствующие в данных цели пропускаются.

Следующая модель получает обстоятельства и все ранее полученные предсказания:

$$\widehat{y}_1 = f_1(x),$$
$$\widehat{y}_2 = f_2(x, \widehat{y}_1),$$
$$\widehat{y}_3 = f_3(x, \widehat{y}_1, \widehat{y}_2).$$

Функция `_augment` добавляет предыдущие предсказания как новые числовые колонки. Например, модель штрафа может использовать уже предсказанный срок лишения права, если соответствующий таргет находится раньше в цепочке. Это способ учитывать статистические связи между компонентами решения; такая связь не доказывает причинного влияния одного наказания на другое.

На каждом шаге обучается тот же градиентный бустинг, что и в independent. Затем он предсказывает текущий таргет на обучающих строках, и эти предсказания передаются следующему оценивателю. В текущем коде передаются именно предсказания на тех же строках, на которых модель обучалась.

На новом деле цепочка выполняется в том же порядке, постепенно накапливая собственные предсказания. Ошибка раннего оценивателя может повлиять на последующие. Дополнительное ограничение состоит в том, что обучающие предсказания обычно точнее предсказаний для новых дел: следующие модели могут чрезмерно доверять этому контексту. Возможное улучшение - формировать обучающий контекст через out-of-fold предсказания. Цепочка не является дискретной цепью Маркова: каждый шаг использует исходные обстоятельства и все предыдущие выходы, а не только последнее состояние.

8.3 Graph

В текущем коде graph представляет собой ансамбль нескольких цепочек, реализованный в `GraphDecisionModel`. Создаются четыре порядка: исходный, обратный и два случайных, воспроизводимых при заданном seed. Каждая цепочка обучается на тех же выбранных обучающих строках, но использует свой порядок таргетов и собственные оцениватели.

Для числового таргета усредняются предсказания цепочек после обратного преобразования в исходные единицы. Для классификационного таргета выбирается наиболее частая предсказанная метка:

$$\widehat{y}_j^{\text{reg}} = \frac{1}{4} \sum_{r=1}^{4} \widehat{y}_j^{(r)},$$

$$\widehat{y}_j^{\text{class}} = \arg\max_k \sum_{r=1}^{4} \mathbf{1}\{\widehat{y}_j^{(r)} = k\}.$$

Здесь r обозначает цепочку, k - возможный класс, а индикатор $\mathbf{1}$ равен единице при выполнении условия. Используется голосование меток, а не усреднение вероятностей. При равном числе голосов текущая реализация выбирает меньшую числовую метку, что для бинарного таргета означает выбор нуля.

Разные порядки позволяют одному таргету в разных цепочках использовать разные предшествующие выходы. Объединение может уменьшать зависимость результата от выбранного порядка, если ошибки цепочек различаются. Однако цепочки используют

одни данные и одно семейство оценивателей, поэтому их ошибки могут быть сильно связаны. Метод требует обучения до четырёх оценивателей на каждый таргет и не гарантирует улучшения. Название graph здесь обозначает ансамбль зависимостей между выходами; графовая нейросеть с передачей сообщений между узлами в коде не реализована. Ограничение обучающего контекста, описанное для chain, сохраняется.

8.4 NLP baseline

Класс `NlpDecisionModel` сначала вызывает `build_text_rows`. Функция записывает непустые признаки в виде токенов «имя_значение», например `c_article_code_128`. При наличии текстовой колонки `circumstances` её содержимое также добавляется. Если в таблице есть только числовые векторы, получается псевдотекст:

$$x \longrightarrow \text{псевдотекст} \longrightarrow \text{TF-IDF-вектор } z \longrightarrow \widehat{\mathbf{y}}.$$

Таким образом, режим не требует слов в исходных данных: строковое представление создаётся из значений признаков. Это категориальный baseline с текстовым способом кодирования, а не автоматически подключённый анализ полного судебного документа.

Используется `TfidfVectorizer`: словные униграммы и биграммы, до 60000 признаков, `min_df=1` и `sublinear_tf=True`. Для токена t, встречающегося в строке i, сначала рассчитывается вес a_{it}, затем нормализуется строка:

$$a_{it} = (1 + \log n_{it}) \left(\log \frac{N+1}{df_t + 1} + 1 \right), \qquad z_{it} = \frac{a_{it}}{\sqrt{\sum_v a_{iv}^2}}.$$

Здесь n_{it} - число вхождений токена в строку, df_t - число обучающих строк с этим токеном, N - размер корпуса, переданного векторизатору. Для отсутствующего токена вес равен нулю; полностью пустая строка остаётся нулевой. Словарь и IDF обучаются только на выбранных обучающих строках. Для новых строк применяется готовое преобразование; неизвестные токены игнорируются. Oversample влияет и на статистики корпуса, поскольку повторённые строки участвуют в обучении TF-IDF.

На каждый бинарный или категориальный таргет отдельно обучается `LogisticRegression`. Для бинарной цели вероятность единицы определяется сигмоидой, а при нескольких классах используются отдельные линейные оценки и softmax:

$$p(y = 1 \mid z) = \frac{1}{1 + \exp(-(w^\top z + b))}, \qquad p(y = k \mid z) = \frac{\exp(w_k^\top z + b_k)}{\sum_\ell \exp(w_\ell^\top z + b_\ell)}.$$

Вектор w содержит обучаемые коэффициенты токенов, b - свободный член. Модель обучается уменьшать логарифмическую потерю с регуляризацией. В коде заданы `max_iter=1000` и `class_weight` со значением `balanced`: балансировка классов таргета включена даже при внешнем режиме `balance=none`. Выбор sigmoid или softmax зависит от количества классов выходного таргета, а не от типа входных признаков.

Для числовых целей используется `Ridge` с `alpha=1.0`. На логарифмированном таргете u_{ij} задача имеет вид:

$$\min_{w_j, b_j} \sum_i s_i \big(u_{ij} - w_j^\top z_i - b_j\big)^2 + \alpha \|w_j\|_2^2.$$

Здесь s_i - вес обучающей строки; без внешнего взвешивания он равен единице. Квадратичный штраф ограничивает величину коэффициентов и стабилизирует модель при

большом числе связанных признаков. Ridge выбран для числового выхода: он может использовать категориальные закономерности, потому что они уже представлены координатами TF-IDF, но сам по себе не является специальным алгоритмом для категорий.

Представление позволяет быстро обучать линейные модели по отдельным значениям признаков и некоторым их сочетаниям. Однако IDF повышает вес редких токенов, а не редких исходов или статей как групп объектов. Биграммы учитывают соседние токены в сформированной строке, а не все возможные взаимодействия признаков. Стандартная токенизация разделяет текст по пунктуации, поэтому дробные числа могут разбиваться на части. Числовая близость значений и их порядок явно не сохраняются. Полезность такого кодирования должна проверяться сравнением с табличными моделями и альтернативными представлениями.

8.5 MLP

Класс `MlpDecisionModel` обучает сеть `MultiTaskNet`: общее представление обстоятельств используется несколькими выходными слоями, по одной на таргет. В отличие от independent, параметры общего блока обновляются под влиянием всех задач:

$$x \longrightarrow \text{стандартизация} \longrightarrow 256 \longrightarrow 128 \longrightarrow 64 \longrightarrow \text{выходные слои таргетов.}$$

Числа обозначают размеры скрытых слоёв. Каждый скрытый блок содержит `Linear`, `BatchNorm1d`, `ReLU` и `Dropout` с вероятностью 0.25. Линейный слой смешивает признаки, BatchNorm нормализует промежуточные активации, ReLU добавляет нелинейность, а Dropout случайно отключает часть активаций во время обучения. При предсказании Dropout выключен.

Входные признаки стандартизируются как $\widetilde{x}_k = (x_k - \mu_k)/\sigma_k$; средние и масштабы рассчитываются по выбранным обучающим строкам, нулевой масштаб заменяется единицей. Регрессионные таргеты после логарифмирования дополнительно стандартизируются. Регрессионный выходной слов выдаёт одно число; классификационная - по одной оценке на класс. Для бинарного таргета с двумя классами также используются две оценки и `CrossEntropyLoss`, а не отдельная sigmoid-выходной слой. При предсказании выбирается класс с максимальной оценкой; числовой выход проходит обратную стандартизацию и обратное логарифмирование.

Для мини-батча B складываются потери всех выходных слоёв:

$$\mathcal{L}_B = \sum_{j \in \mathcal{C}} \frac{1}{|B|} \sum_{i \in B} -\log p_j(y_{ij} \mid x_i) + \sum_{j \in \mathcal{R}} \frac{1}{|B|} \sum_{i \in B} (\widehat{v}_{ij} - v_{ij})^2.$$

Множества $\mathcal{C}$ и $\mathcal{R}$ содержат классификационные и регрессионные таргеты; v_{ij} - преобразованный и стандартизированный числовой таргет. Каждый выходной слой входит в сумму с коэффициентом 1. Используется AdamW: скорость обучения 10^{-3}, `weight_decay` 10^{-4}, размер батча 128. Число эпох задаётся параметром `epochs`; прогресс показывает `tqdm`. Выбор лучшей эпохи по validation в текущем цикле не реализован.

Метод `_prepare_targets` вызывается для всей таблицы до выделения обучающих тензоров. Поэтому средние и масштабы регрессионных таргетов, а также наборы классов определяются с участием validation и test. Градиенты рассчитываются только

по обучающим строкам, но использование отложенных таргетов при подготовке является утечкой информации. Для корректного benchmark эти параметры необходимо определять только на train и затем применять к остальным выборкам.

Общее представление может быть полезно, если разные санкции зависят от общих обстоятельств. Однако задачи могут конкурировать за параметры, а одинаковые коэффициенты потерь не гарантируют одинакового влияния на обучение. Сеть требует настройки регуляризации и объёма обучения. Её выходные слои связаны общими признаками, но предсказание одного выходного слоя напрямую не подаётся в другую и согласованность сочетания наказаний не гарантируется.

Во всех режимах `apply_balance` изменяет только обучающие индексы или веса. Для independent, chain и graph веса передаются в бустинг; при наличии внешних весов для бинарного таргета дополнительно применяется множитель N_0/N_1 к положительным примерам. При `none` и `oversample` эта дополнительная коррекция не включается. В NLP внешние веса передаются оценивателям, причём у логистической регрессии дополнительно действует `class_weight=balanced`. В MLP внешние веса используются через `WeightedRandomSampler`: объекты выбираются с вероятностями, пропорциональными весам, вместо прямого умножения потерь на веса.

9 Балансировка классов и статей

Поддерживаются режимы:

Режим	**Что делает**
`none`	обучает на исходном распределении
`article_weights`	даёт больший вес редким статьям
`strata_weights`	даёт больший вес редким стратам статья–регион–год
`oversample`	повторяет редкие страты в train

Эффект балансировки зависит от модели: в приведённых далее таблицах есть как снижения score, так и небольшие улучшения. Веса могут ухудшать общий score по нескольким причинам: при хорошем разделении статей через `c_article_code` и регуляторные признаки дополнительное усиление редких наблюдений не обязательно полезно; при более шумных таргетах редкой статьи оно усиливает и этот шум; улучшение редкой статьи может не компенсировать ухудшение частой в общей метрике. Это возможные объяснения, а не установленные причины данного эксперимента. Для анализа дисбаланса следует совместно смотреть `test_decision_score`, `test_balanced_article_score`, `test_article_score_gap` и `by_article`.

10 Метрики качества

10.1 Target score

Для каждого таргета считается своя метрика.

Для binary/class:

$$\texttt{target_score} = \text{macro-F1}$$

Macro-F1 усредняет F1 по классам и меньше подыгрывает частому классу.

Для regression:

$$\texttt{target_score} = \max\left(0,\ 1 - \frac{\mathrm{MAE}}{\mathrm{scale}}\right),$$
$$\mathrm{scale} = \mathrm{mean}(|y_{\mathrm{true}}|) + \mathrm{std}(y_{\mathrm{true}}) + 1.$$

Чем ближе `target_score` к 1, тем лучше.

10.2 `test_decision_score`

Главная общая метрика:

$$\texttt{test_decision_score} = \frac{1}{n}\sum_{j=1}^{n} S_j,$$

где S_j - score таргета j, а n - число оцениваемых таргетов.

Она показывает среднее качество по всему вектору судебного решения на test split. Ограничение: если в векторе много почти всегда нулевых таргетов, общий score может выглядеть слишком высоким.

10.3 `test_balanced_article_score`

Метрика для дисбаланса между статьями:

$$\texttt{test_balanced_article_score} = \mathrm{mean}\big(\texttt{score}(\mathrm{article}_i)\big)$$

Для каждой статьи score заново рассчитывается по её строкам и усредняется по таргетам. Затем результаты статей усредняются с равными весами независимо от числа строк. В регрессионных компонентах масштаб также рассчитывается внутри соответствующей статьи.

10.4 `test_substantive_score`

Среднее качество только по материальным санкциям: `d_imprisonment_epochs`, `d_license_revocation_epochs`, `d_mandatory_work_hours`, `d_forced_work_months`, `d_corrective_work_months`, `d_restriction_freedom_months`, `d_fine_rub`. Это более содержательная метрика, чем общий score, потому что она меньше зависит от пустых бинарных таргетов.

10.5 `test_article_score_gap`

Разрыв между лучшей и худшей статьёй:

$$\texttt{test_article_score_gap} = \max_i S_i^{\mathrm{art}} - \min_i S_i^{\mathrm{art}},$$

где S_i^{art} - score статьи i. Чем меньше значение, тем ровнее модель работает по статьям.

10.6 `legally_constrained_decision_score`

Score после регуляторной постобработки предсказаний, рассчитанный относительно тех же целевых значений. Снижение означает, что корректировки увеличили ошибку по размеченным решениям. Частоту изменений показывает `legal_constraint_report`, а не разница score сама по себе. Причиной расхождения могут быть ошибки таргетов, неверная дата или часть статьи, неучтённый исход либо исключение, а также неполнота правил.

11 Текущие результаты

11.1 Полный сохранённый baseline результаты для independent

Конфигурация:

Параметр	Значение
method	independent
balance	none
rows	91 827
features	52
targets	9
train / val / test	64 284 / 13 774 / 13 769

Test-метрики:

Метрика	Значение
`test_decision_score`	0.967720
`test_balanced_article_score`	0.965289
`test_substantive_score`	0.903161
`test_article_score_gap`	0.044436
`test_legally_constrained_decision_score`	0.862982

Качество по статьям:

Статья	Test support	Decision score	Substantive score
12.8	10 591	0.987507	0.962521
264.1	3 178	0.943071	0.829213

Вывод: `independent + none` является сильным табличным baseline. По общей метрике качество высокое, но `substantive_score` по 264.1 ниже, чем по 12.8. Это указывает, что материальные санкции по 264.1 сложнее и требуют отдельного анализа.

12 Основные проблематики

12.1 Разная природа статей

Одинаковые признаки не означают одинаковую функцию решения. Пример: `c_recidivism_count = 1`. Для 12.8 это может быть административно значимое обстоятельство. Для 264.1 повторность связана с самой природой уголовного состава и может иначе влиять на наказание.

Что можно развивать: отдельные модели по статьям; общий encoder + article-specific heads; mixture of experts; Group DRO; temporal/group split.

12.2 Регуляторная согласованность

Сырой ML-output может быть статистически близким, но юридически недопустимым.

Что можно потенциально развивать: точнее кодировать правовые режимы; строить отдельные тесты на юридическую допустимость предсказаний.

12.3 Работа с реальным текстом

Текущий nlp baseline использует pseudo-text из числовых признаков, а не полный текст судебного акта.

Для дальнейшего развития: TF-IDF/BM25 по реальному тексту; sentence embeddings; transformer encoder; hierarchical transformer для длинных документов; retrieval-augmented prediction; hybrid tabular + text model.

13 Экспериментальное тестирование методов и способов балансировки

Приведены эксперименты с `court_decision_baseline.py`. Ниже сохранены их конфигурация и результаты;

13.1 Конфигурация эксперимента

Для тестирования использовались два основных набора:

Источник	**Содержание**
`koap_vectors/12.8 Сургут/overall/vectors.csv`	дела по статье 12.8 КоАП
`koap_vectors/264.1 суды общей инстанции/vectors.csv`	дела по статье 264.1 УК РФ

Общая конфигурация:

Параметр	**Значение**
Количество строк	91 827
Число входных признаков `c_*`	52
Число целевых признаков `d_*`	9
Train	64 284
Validation	13 774
Test	13 769
Разбиение	70% / 15% / 15%
Seed	42
Ключ стратификации	статья : регион : год
MLP epochs	20

Пересечений индексов между train, validation и test нет.

Проверялись методы: `independent`, `chain`, `graph`, `nlp`, `mlp`.

Для каждого метода проверялись режимы балансировки: `none`, `article_weights`, `strata_weights`, `oversample`.

Всего проведено 20 основных экспериментов и два дополнительных контрольных запуска после очистки данных.

13.2 Состав и дисбаланс данных

Распределение дел по статьям:

Статья	Количество	Доля
12.8 КоАП	70 615	76.90%
264.1 УК РФ	21 207	23.1%

Распределение содержательных таргетов:

Таргет	Ненулевых строк	Доля ненулевых
`d_imprisonment_epochs`	16 810	18.31%
`d_license_revocation_epochs`	49 377	53.77%
`d_fine_rub`	1 213	1.32%

Остальные шесть таргетов преимущественно не содержат данных согласно текущим постановлениям: `d_conditional`, `d_travel_ban`, `d_reconciliation`, `d_costs_on_state`, `d_costs_amount_rub`, `d_self_recusal`.

Основной содержательный показатель - `test_substantive_score`, рассчитанный по: `d_imprisonment_epochs`, `d_license_revocation_epochs`, `d_fine_rub`.

13.3 Сравнение методов без балансировки

Метод	Decision	Balanced art.	Art. gap	Substantive	Время, с
Independent	0.967720	0.965289	0.044436	0.903161	94.1
Chain	0.967724	0.965294	0.044432	0.903172	31.8
Graph	0.967720	0.965289	0.044445	0.903159	118.7
NLP	0.967565	0.965114	0.044424	0.902694	34.8
MLP	0.967614	0.965196	0.044499	0.902842	165.5

`chain` формально получил лучший результат среди режимов без балансировки, но разница с `independent` составляет только 0.000004. Практически методы эквивалентны.

`graph` не улучшил качество относительно `independent`, но требует обучения четырёх цепочек.

`nlp` оказался немного хуже. Это ожидаемо, поскольку текущий NLP baseline использует pseudo-text вида `feature=value`, а не исходный текст судебного акта.

`MLP` также не дал преимущества без балансировки и требует значительно больше вычислений.

13.4 Полная матрица балансировки

Значения `test_decision_score`:

Метод	None	Article weights	Strata weights	Oversample
Independent	0.967720	0.967708	0.967701	0.967763
Chain	0.967724	0.967737	0.967660	0.967745
Graph	0.967720	0.967718	0.967665	0.967733
NLP	0.967565	0.967563	0.967383	0.967355
MLP	0.967614	0.967642	0.967699	0.967803

Лучшие конфигурации каждого метода:

Метод	Balance	Decision	Substantive	Balanced art.	Gap	Время
Independent	Oversample	0.967763	0.903290	0.965401	0.044308	128.2 с
Chain	Oversample	0.967745	0.903235	0.965340	0.044233	173.9 с
Graph	Oversample	0.967733	0.903199	0.965352	0.044328	297.9 с
NLP	None	0.967565	0.902694	0.965114	0.044424	34.8 с
MLP	Oversample	0.967803	0.903550	0.965409	0.044097	1 482.8 с

Формально лучший результат получен для `method = mlp`, `balance = oversample`: `test_decision_score = 0.967803`, `test_substantive_score = 0.903550`. Однако улучшение относительно `independent + none` составляет: decision score +0.000083, substantive score +0.000389 что при скорости обучения не такой уж и хороший прирост.

13.5 Результаты по статьям

Сравнение исходного baseline и лучших конфигураций:

Конфигурация	12.8 decision	12.8 subst.	264.1 decision	264.1 subst.
Independent + none	0.987507	0.962521	0.943071	0.829213
Independent + oversample	0.987555	0.962664	0.943246	0.829739
MLP + oversample	0.987458	0.962522	0.943361	0.830196

По статье 12.8 качество существенно выше, чем по статье 264.1. Oversample и MLP немного улучшают результат для 264.1, но прирост `substantive_score` относительно основного baseline составляет меньше 0.001. Это подтверждает, что проблема 264.1 связана не только с количеством строк: статьи имеют разную юридическую и статистическую природу, поэтому простое изменение весов не устраняет разрыв.

13.6 Результаты способов балансировки

Article weights. Вес каждой статьи обратно пропорционален её размеру. В train получены веса:

Статья	Средний вес
12.8	0.4335
264.1	1.4431

Strata weights. Веса рассчитываются по стратам `статья : регион : год`. Максимальный вес составил 905.41, стандартное отклонение весов - 17.35. Режим иногда уменьшает `article_score_gap`, но обычно снижает общий и содержательный score. Например, для NLP: `none: substantive = 0.902694`, `strata_weights: substantive = 0.902149`.

Oversample. Oversample увеличил train с 64 284 до 594 341 строк (коэффициент 9.25). После oversample распределение в train:

Статья	Строк в train
12.8	259 501
264.1	318 098

Этот режим балансирует страты, а не непосредственно статьи. В результате 264.1 ста-

новится больше 12.8, а одни и те же наблюдения многократно повторяются. Oversample дал небольшое улучшение для табличных моделей и MLP, но ухудшил NLP (`none: 0.967565` vs `oversample: 0.967355`). Для MLP `substantive_score` вырос с 0.902842 до 0.903550, но время обучения увеличилось с 2.8 до 24.7 минуты.

13.7 Итоговый вывод

На текущих данных оптимальным MVP остаётся `method = independent`, `balance = none`. Причины:

- `chain` не даёт значимого прироста;
- `graph` заметно дороже и не улучшает качество;
- NLP по pseudo-text немного хуже;
- MLP с oversample формально лучший, но выигрыш меньше 0.0001 по общей метрике при значительном росте времени;
- способы балансировки не устраняют разрыв между 12.8 и 264.1;
- основное ограничение качества связано с данными, а не с архитектурой модели.

Наблюдаемые различия между конфигурациями малы. Чтобы отличить устойчивое улучшение от вариативности эксперимента, нужны повторные разбиения или запуски с несколькими seed и доверительные интервалы. Проверка разметки санкций, смешения статей и соответствия регуляторных профилей остаётся отдельным направлением работы.